“Sentinel:
True Identity Engine”

A PROJECT WORK REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institution Under VTU)

In fulfillment of the requirements for major project work,
Seventh semester

Bachelor of Engineering in Computer Science & Engineering

Submitted By

Suhas U (4NI22CS222)

Under The Guidance Of

Ms. Zaiba Farheen
Assistant Professor
Department of CS&E, NIE
Mysuru-570008

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THE NATIONAL INSTITUTE OF ENGINEERING
(An Autonomous Institution Under VTU)
Mananthavadi Road, Mysuru- 570008

2025-2026

THE NATIONAL INSTITUTE OF ENGINEERING
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

Department of Computer Science & Engineering

CERTIFICATE

This is to Certify that the project work entitled **"Sentinel: True Identity Engine"** is a bonafide work carried out by Suhas U (4NI22CS222) in fulfillment for major project work, seventh semester, Computer Science and Engineering, The National Institute of Engineering (Autonomous under VTU) during the academic year 2025–2026. It is certified that all corrections and suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the department library. The major project work report has been approved in fulfillment as per academic regulations of The National Institute of Engineering, Mysuru.

Signature of Guide	**Signature of HOD**	**Signature of Principal**
---	---	---
Ms. Zaiba Farheen **Assistant Professor** Dept. of CSE NIE, Mysuru	**Dr. Anitha R** **Professor and Head** Dept. of CSE NIE, Mysuru	**Dr. Rohini Nagapadma** **Principal** Dept. of CSE NIE, Mysuru

Name of the Examiners	**Signature and Date**
---	---
---	---

ABSTRACT

Ensuring the integrity of electoral rolls is a key challenge for any democracy, as duplicate or fraudulent entries threaten fair elections. This threat undermines public trust and skews the representation of the electorate, making the achievement of "one person, one vote" impossible. Traditional de-duplication methods—manual review or simple hashing—are slow, costly, or ineffective against minor data variations and intentional deceit, necessitating a modern, resilient approach.

This project, Sentinel: True Identity Engine, tackles this by combining automated facial recognition with a "human-in-the-loop" verification process. Built using HTML, CSS, and JavaScript with the face-api.js library, it efficiently runs a Convolutional Neural Network (CNN) directly in the browser. This client-side architecture ensures secure and decentralized face matching, leveraging the power of deep learning without compromising data privacy.

The system features two distinct user roles: The Admin uploads voter data (CSV + images), and the system generates 128-point facial descriptors for comparison. Instead of auto-rejecting potential matches based on an arbitrary score, it intelligently creates a Verification Case grouping potential duplicates for human scrutiny. The Verifier then reviews each case side-by-side, analysing the evidence before definitively deciding which entry is original or duplicate.

With role separation, robust audit logging, and a transparent case-based workflow, Sentinel ensures accuracy, transparency, and fairness in the critical process of roll management. It successfully turns complex biometric de-duplication into a reliable, human-centered process, restoring confidence in electoral data integrity.

ACKNOWLEDGEMENT

We sincerely owe our gratitude to all the persons who helped and guided us in completing this major project.

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts. First and foremost, we would like to thank our beloved principal **Dr. Rohini Nagapadma** for being the patron and the beacon light for this project.

We would like to express our sincere gratitude to our HOD, **Dr. Anitha R**, Department of CSE, NIE, for her relentless support and encouragement.

It gives us immense pleasure to thank our guide, **Ms. Zaiba Farheen**, Assistant Professor, Department of CSE, NIE, for her valuable suggestions and guidance during the process of the project and for having permitted us to pursue work on the subject.

Suhas U (4NI22CS222)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter - 1

INTRODUCTION

1.1 Overview

The integrity of a voter registration list is indeed a fundamental pillar of a democratic election, serving as the definitive roster of eligible voters. Its accuracy is paramount because it underpins the principle of legitimacy and the basic standard of "one person, one vote."

A significant and persistent challenge in managing electoral rolls is the presence of duplicate or fraudulent entries. These inaccuracies create several interrelated problems:

- Bloated Voter Lists: Duplicates artificially inflate the number of registered voters. This complicates election logistics, such as budgeting for ballots and staffing polling stations, often leading to unnecessary expenditure.
- Complicated Logistics: Inaccurate data makes it harder to plan efficiently, potentially causing long queues or shortages of materials in areas where the actual number of unique voters differs significantly from the registered count.
- Erosion of Public Trust: Most critically, the presence of duplicates and, especially, fraudulent entries, erodes public trust in the electoral process. The perception that the lists are flawed or susceptible to manipulation can lead to post-election disputes and a general skepticism about the fairness and outcome of the vote.
- Vulnerability to Fraud: Fraudulent entries can facilitate illegal voting, either through impersonation or by providing cover for other forms of manipulation.

The rise of digital registration has been a double-edged sword: while it has made the registration process more accessible and efficient for genuine citizens, it has also opened new avenues for systemic error and intentional duplication. The ease of data entry means that minor variations in names, dates of birth, or addresses are simple to implement, effectively bypassing older, text-based de-duplication checks and making the detection of a single individual registered multiple times a significant technical hurdle.

1.2 Objectives

The Sentinel project is built around five core objectives to ensure the highest degree of security, fairness, and transparency in electoral roll de-duplication:

- Dual-Role Web Application Design:
    - To design and deploy a secure web application featuring two mandatory, distinct user roles: Admin and Verifier.
    - This architecture enforces a critical Segregation of Duties (SoD), where the Admin manages the data upload and system configuration, but only the Verifier can make final, binding decisions on voter records, preventing single-point compromises.
- Biometric Processing Engine Implementation:
    - To implement a robust facial recognition engine utilizing a Convolutional Neural Network (CNN).
    - This engine processes voter images to generate and compare unique 128-point facial descriptors. This replaces ineffective text-based checks with biometric certainty, ensuring accurate identification of duplicate identities regardless of minor data changes.
- Intelligent Case-Based Workflow Creation:
    - To establish an intelligent workflow that automatically clusters all potential duplicates (new and existing) into a single Verification Case.
    - By creating cases instead of auto-rejecting, the system ensures due process and avoids the high risk of false positives. This shifts the process from automation to informed human review.
- Intuitive Verifier Adjudication Dashboard:
    - To build a specialized dashboard enabling the Verifier to perform manual adjudication of cases.
    - The dashboard presents conflicting entries side-by-side with both image and data, allowing the Verifier to confidently designate one record as the Original and others as Duplicates, thereby making a final, auditable judgment.

- Accountability via Audit Logging:
    - Objective: To implement a mandatory, secure Audit Log (history) to track every verification decision.
    - Purpose: This ensures complete transparency and accountability by creating an immutable record of who made which decision, when, and for which voter record. This log is essential for external audits and public confidence.

1.3 Need for Project

Traditional methods for voter de-duplication are demonstrably no longer sufficient to uphold the integrity of modern, large-scale electoral rolls. The limitations of legacy systems create critical vulnerabilities that necessitate a definitive technological shift:

- Manual Review: The Crisis of Scale and Error
    - Manual review involves staff visually inspecting millions of records, often spanning across multiple paper documents or disparate digital databases. This process is exceedingly slow, labor-intensive, and financially costly, rendering it unscalable for nations with large or rapidly growing populations.
    - More critically, human reviewers are inherently prone to error when faced with long hours and overwhelming data. They struggle to link two visually similar but textually distinct records, especially when the intent is deliberate fraud, not accidental duplication.
- Simple Data Matching: Easily Defeated by Data Variations
    - Methods relying on simple data matching (such as comparing names, dates of birth, or partial addresses) are fundamentally brittle. They fail instantly when faced with common issues like misspellings, typographical errors, or minor data variations, whether accidental or intentionally introduced to bypass the system.
    - A single individual can register multiple times using slightly altered personal information, and these basic algorithms will register them as unique voters, allowing fraudulent registrations to persist unchecked in the system.

- Cryptographic Hashing: A Flawed Technical Solution for Images
    - While cryptographic hashing (like SHA-256) is excellent for verifying the integrity of text files, its application to image de-duplication is a flawed technical solution. Hashing only detects byte-for-byte identical files.
    - The method fails entirely to match two photos of the same person if those photos were taken from slightly different angles, under varying lighting conditions, or even if the same photo was simply saved in a different file format (e.g., JPEG vs. PNG). This makes it useless for real-world de-duplication where image variety is the norm.

1.4 Existing Systems and Drawbacks

Current methods used to manage electoral roll integrity fall into three main categories, each with fundamental flaws that prevent them from offering a reliable and scalable solution.

- Manual Verification: Election officials, often relying on paper or unintegrated digital databases, manually compare new registration forms against existing records. This labor-intensive task involves visually cross-referencing names, addresses, and sometimes photos across massive lists.

  Drawbacks:
    - Scale and Cost Inefficiency: This method is extremely slow and scales poorly. For a population of even a few million, manual review is prohibitively costly and requires immense labor and time, often delaying the final certification of voter lists.
    - High Error Rate and Oversight: The process is highly susceptible to human error and oversight. Reviewers fatigue easily, making it difficult to spot subtle variations or intentional fraudulent registrations. This lack of rigorous oversight is precisely where sophisticated duplications can slip through undetected, compromising the integrity of the entire roll.

- Simple Hashing (Cryptographic Hashing for Images): The system attempts a technical shortcut by storing a unique cryptographic hash (a short string of text) generated from a voter's photo file. When a new photo is uploaded, its hash is generated and compared against the stored hashes.

Drawbacks:

- Useless for Biometric Matching: Simple hashing is only designed to verify file integrity, not visual similarity. It fails completely if the photo files are not byte-for-byte identical.
- Hypersensitivity to Data Change: Even a trivial, invisible change in image data—such as a 1% change caused by resizing, minor compression differences, a conversion from JPG-to-PNG, or even the same photo saved with a different timestamp—results in a 100% different hash. This means the system cannot match two slightly different photos of the same person, making it useless for real-world biometric de-duplication.

- Fully Automated AI Rejection (No Human Oversight): This represents the next-generation approach, where a system uses advanced facial recognition to find a match but then immediately and automatically rejects the new entry or flags the old one without human intervention.

  Drawbacks:

  - High Risk of Disenfranchisement: While this approach is fast, it is dangerous and unjust. AI models are powerful but are not perfect and can produce "false positives" (incorrect matches) due to poor image quality, lighting variations, or demographic bias inherent in some models.
  - Lack of Due Process: An incorrect match triggered by the AI could automatically and unfairly disenfranchise a legitimate voter. Since there is no human-in-the-loop, the voter is denied any opportunity for appeal or review, violating fundamental principles of electoral fairness and transparency. This is precisely the critical gap that Sentinel's governance model is designed to fill.

1.5 Proposed System and its Advantages

This project proposes a **Case-Based, Human-in-the-Loop Biometric Detector** known as **Sentinel**. It is a secure, modern **web application** that intelligently resolves the conflict between automated efficiency and guaranteed fairness by combining powerful AI processing with a robust manual verification workflow.

The core advantages of this proposed system directly address the systemic failures of traditional and purely automated approaches:

1. High Accuracy through Deep Learning

- Mechanism: Sentinel achieves superior accuracy by utilizing a Convolutional Neural Network (CNN), implemented efficiently through the face-api.js library. This is true facial recognition, not simple hashing. The CNN processes the image and extracts a 128-point face embedding (descriptor)—a unique, compact, mathematical representation of the face.
- **Advantage**: This method allows the system to successfully match two different photos of the same person, even when they present high variance due to different angles, changes in lighting, varying facial expressions, or even minor changes over time. This robust feature extraction capability ensures that real duplicates are identified where simple systems would fail.

2. Guaranteed Fairness and Protection Against False Positives

- Mechanism: The system adheres to a strict principle: it never automatically rejects a voter. The AI's role is strictly limited to detection. When a potential match is found (a similarity score is high), the process always defers to a human reviewer.
- **Advantage**: By automatically creating a "Verification Case," Sentinel ensures that a qualified Verifier makes the final, common-sense judgment. This "human-in-the-loop" model acts as a critical safeguard, protecting legitimate voters from potential "false positives" or errors generated by the AI model, thereby ensuring due process and avoiding the political risk of unfair disenfranchisement.

3. Secure Separation of Roles (Segregation of Duties)

- Mechanism: The system architecture is explicitly designed to enforce a strict Separation of Duties (SoD) through its dual-role access control.
- **Advantage**: The Admin role is strictly a data-entry and system monitoring operator focused on uploading files and tracking workflow. The Verifier is a trusted adjudicator focused on making integrity decisions. Crucially, the Admin cannot approve cases, and the Verifier cannot upload or delete bulk data. This strict boundary prevents internal collusion, single-point failure, and ensures that sensitive decision-making is restricted to audited, authorized personnel.

4. Accountability and Transparency through Audit Logging

- Mechanism: The system integrates a robust, non-editable Admin history log that captures every critical transaction and decision within the platform.
- Advantage: This feature creates a clear and immutable audit trail. It answers the fundamental governance questions: who (which Verifier, identified by a unique ID) resolved what (which specific duplicate case), and when (with a precise timestamp). This complete record is absolutely critical for maintaining the integrity of the database, providing evidence for public scrutiny, legal challenges, and mandatory external audits.

Chapter 2

LITERATURE SURVEY

The paper titled "Research on Blockchain Electronic Voting System Based on Face Recognition and Deep Fake Face Detection" by Tolegen et al. (IJIRSS, 2025) proposes a high-security e-voting system that integrates facial recognition with deepfake detection to ensure electoral integrity. The framework uses advanced deep learning models such as Vision Transformers (ViT) and ResNet for voter authentication and employs blockchain for immutable, auditable vote storage [1].

Key contributions of this work include:

- Integrating facial recognition with deepfake detection to prevent identity spoofing.
- Achieving 97.36% authentication accuracy using deep learning models.
- Using blockchain and smart contracts for transparent and tamper-proof storage.

Relevance to our project: This work validates our use of facial recognition as a state-of-the-art solution. It also highlights a future improvement: incorporating deepfake detection to protect against modern spoofing attacks, which our current prototype does not yet address.

The paper titled "E-Voting System Using Blockchain and Face Recognition" by Bagal et al. (IRJET, 2024) presents an e-voting framework that employs Dlib's ResNet-based model to generate 128-dimensional facial encodings, which are compared using Euclidean distance for identity matching [2].

Key contributions of this work include:

- Representing faces as 128-dimensional numerical vectors.
- Using Euclidean distance to calculate similarity between face vectors.
- Confirming identity matches below a set similarity threshold.

Relevance to our project: This research directly supports our chosen approach. Our system uses face-api.js with a ResNet model to generate 128-d descriptors and applies a 0.5 distance threshold—confirming that our design aligns with modern, standard facial recognition practices.

The paper titled "Application of Human-in-the-Loop Hybrid Augmented Intelligence Approach in Security Inspection Systems" (Frontiers, 2025) argues against fully autonomous AI for critical security operations, promoting a hybrid human-AI collaboration model [3].
Key contributions of this work include:

- Warning that full automation can cause false positives and reduced human alertness.
- Advocating for hybrid decision-making, combining human oversight with AI analysis.
- Emphasizing that removing human judgment introduces systemic risks.

Relevance to our project: This directly supports our architecture. Our "Verifier" role serves as the human-in-the-loop, reviewing AI-flagged duplicates before confirmation, ensuring fairness and accountability—precisely what this paper endorses as best practice.

The paper titled "Smart Elections or Rigged Algorithms: The Rise of Artificial Intelligence in Electoral Governance in Southeast Asia" by Syafhendry et al. (2025) examines the real-world implementation of AI technologies in election systems [4].
Key contributions of this work include:

- Highlighting Indonesia's biometric voter ID (e-KTP) and facial recognition system.
- Showing that AI can reduce fraud and improve voter verification accuracy.
- Noting challenges such as privacy concerns and enforcement consistency.

Relevance to our project: This real-world case study validates the relevance of our work. Governments are already deploying similar technology to prevent voter duplication and fraud, demonstrating the practical applicability of our system.

The paper titled "The Threat of Artificial Intelligence to Elections Worldwide: A Review of the 2024 Landscape" (World Journal of Advanced Engineering Technology and Sciences, 2024) surveys the global impact of AI on election systems [5].
Key contributions of this work include:

- Discussing Brazil's 2020 trial of AI-powered facial recognition for voter ID verification.
- Emphasizing reduced impersonation risk and streamlined verification.
- Warning of issues like algorithmic bias and data security vulnerabilities.

Relevance to our project: This reinforces the need for human oversight and bias mitigation in AI-driven electoral systems. Our "Verifier" and case-based review process directly address the fairness and accountability concerns raised in this paper.

The paper titled "Research and Analysis of Facial Recognition Based on FaceNet, DeepFace, and OpenFace" (ITM Web of Conferences, 2025) reviews key deep learning models used for generating facial embeddings in recognition systems [6].

Key contributions of this work include:

- Explaining how models like FaceNet and DeepFace convert facial features into high-dimensional embeddings.
- Discussing the triplet-loss mechanism for optimizing embedding spaces.
- Comparing performance trade-offs between different recognition frameworks

Relevance to our project: This provides theoretical grounding for our use of face-api.js, which follows the same embedding principle. The paper confirms that generating high-dimensional descriptors is the foundation of modern facial recognition technology.

The paper titled "Voter Authentication Using Enhanced ResNet50 for Facial Recognition" by Halidou et al. (MDPI, 2025) introduces an advanced facial recognition model that combines an enhanced ResNet50 architecture with Additive Angular Margin Loss (ArcFace) for highly accurate voter verification [7].

Key contributions of this work include:

- Introducing ArcFace loss to maximize separation between facial identities.
- Achieving 99.85% facial recognition accuracy using ResNet50 + ArcFace.
- Employing MTCNN for robust face detection before recognition.

Relevance to our project: This paper provides a clear path to enhance our model. By replacing our current SsdMobilenetv1 with a ResNet50 + ArcFace combination, we could significantly improve the system's matching accuracy and robustness.

Chapter 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1. Hardware Requirements

- CPU: Intel Core i5 / AMD Ryzen 5 or better.
- RAM: 8 GB minimum.
- Storage: 100 MB of free space (for browser cache).
- Network: A stable internet connection

3.2. Software Requirements

- Operating System: Windows 10/11, macOS or Linux.
- Web Browser: A modern, up-to-date browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) with JavaScript enabled.
- Core Library: face-api.js.
- Development Environment: Visual Studio Code.

3.3. Functional Requirements

- FR-1: Authentication: The system must provide two distinct login portals for "Admin" and "Verifier" roles.
- FR-2: Admin CSV Upload: The Admin user must be able to upload a .csv file containing voter metadata (ID, name, age, state, image_filename).
- FR-3: Admin Image Upload: The Admin user must be able to upload a batch of multiple image files (.jpg, .png).
- FR-4: AI Processing: The Admin must have a "Process & Check Duplicates" button that, when clicked, initiates the facial recognition pipeline.
- FR-5: Parallel Processing: The system must process all uploaded images in parallel to minimize waiting time.
- FR-6: Case Generation: If a new image matches an existing one (with a Euclidean distance < 0.5), the system must create a "Verification Case" and flag *all* involved voters (new and old).
- FR-7: Verifier Queue: The Verifier dashboard must display a queue of all pending "Verification Cases."

- FR-8: Case Review: The Verifier must be able to see all conflicting voter images and data side-by-side within a single case card.
- FR-9: Case Resolution: The Verifier must be able to click a "Mark as Original" button on exactly one voter in the case.
- FR-10: Database Update: On resolution, the system must update the "database" (JS array) to mark the selected voter as "verified" and all others in the case as "duplicate."
- FR-11: Admin History Log: The Admin dashboard must display a read-only audit log of all resolved cases, showing the Verifier's username, the Case ID, and a timestamp.

3.4. Non-Functional Requirements

- Accuracy: The system must accurately match photos of the same person from different angles, lighting, and expressions.
- Reliability (Fairness): The system must *never* auto-delete or auto-reject a voter. Every potential duplicate must be escalated to a human Verifier.
- Performance: The UI must remain responsive. A loading spinner must be shown during the AI processing, which should complete in a reasonable timeframe.
- Usability: The UI for both roles must be clean, intuitive, and responsive. Images in the review queue must be clearly visible and not cropped (object-fit: contain).
- Security: Roles must be strictly separated. An Admin cannot access the Verifier's URL/page, and vice-versa.
- Modularity: The code must be split into separate, logical files (index.html, style.css, script.js) for maintainability.

Chapter 4

SYSTEM DESIGN AND ANALYSIS

4.1 High Level Design

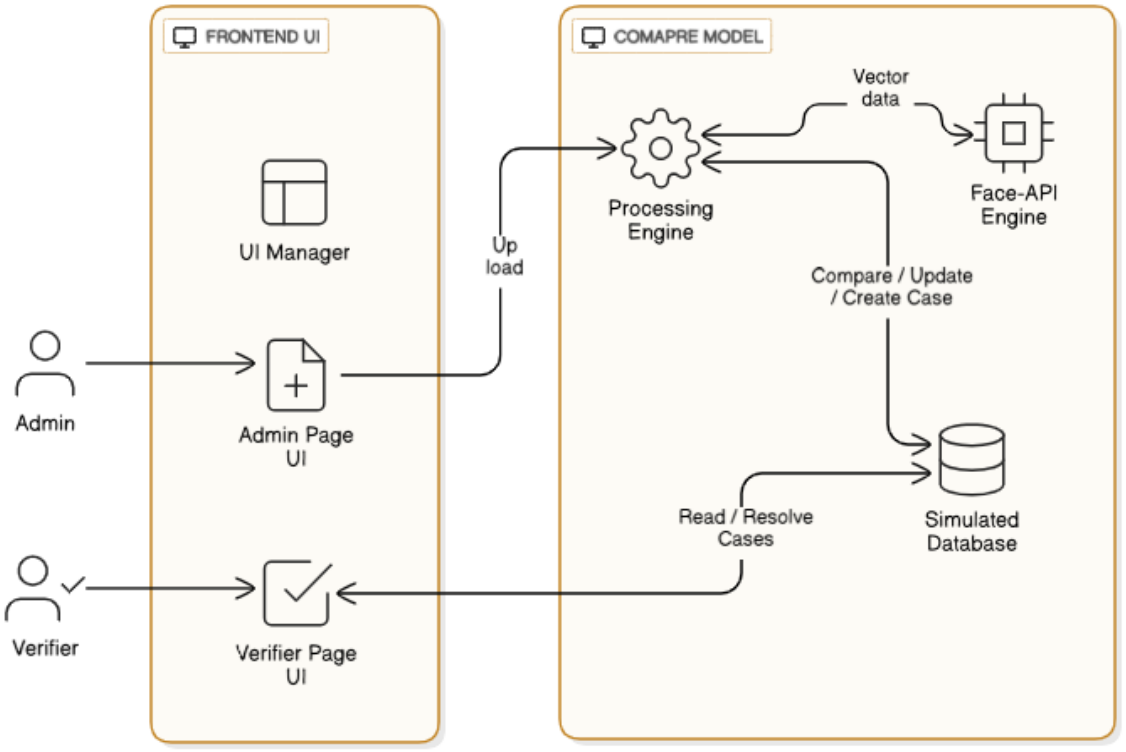


Fig 4.1: High - Level System architecture of Sentinel

The Figure 4.1 shows how the system is designed as a client-side, single-page application (SPA) with two distinct user roles that interact with a central processing engine and a simulated database.

- Admin (User): Interacts with the Admin Page UI to upload CSV data and Image files. They initiate the processing and view the final history log.
- Verifier (User): Interacts with the Verifier Page UI to view pending cases and submit a resolution.
- Application Logic (script.js):
    - UI Manager: Shows/hides the Login, Admin, and Verifier pages.
    - Face-API Engine: The core component. It loads the AI models and exposes functions to generate face descriptors.

- Processing Engine: Handles the parallel processing of images, compares descriptors and creates cases.
- Simulated Database (JS Arrays):
    - voterDatabase: The "master list" of all voter records, including their status (verified, duplicate, flagged).
    - DuplicateFlags: The "work queue" for the verifier, storing the active cases.
    - verificationHistory: A read-only log of all resolved cases.

4.2 Low Level Design

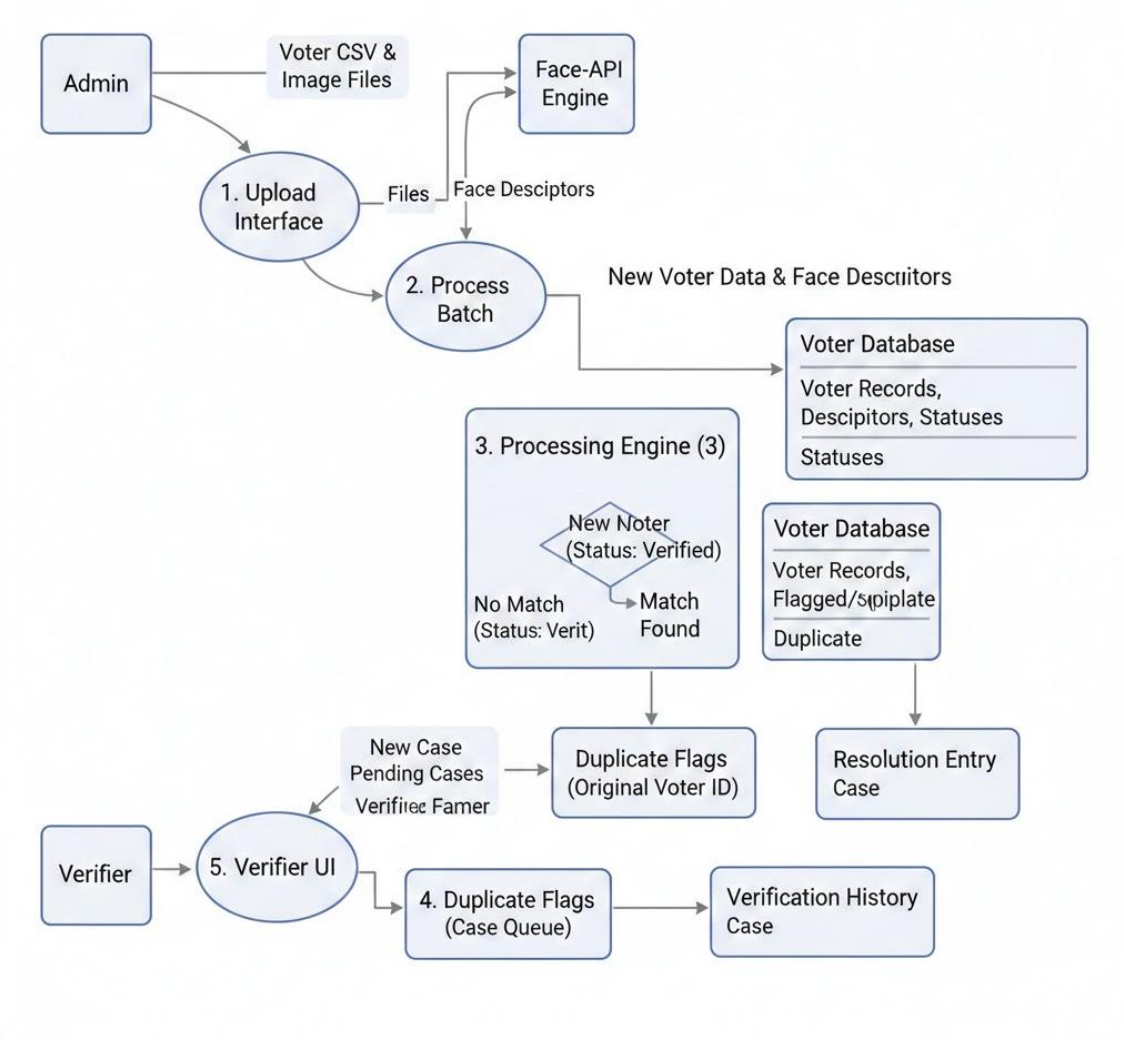


Fig 4.2: Low - Level System architecture of Sentinel

The Figure 4.2, Low-Level Design diagram focuses on the detailed interactions and workflows within specific components of the system:

- Admin provides Voter CSV and Image Files to the Upload Interface.
- The Process Batch function sends the images to the Face-API Engine (in parallel), which returns Face Descriptors.
- The Processing Engine compares descriptors to the voterDatabase.
- If no match, the new voter is added directly to the voterDatabase as "Verified."
- If a match is found, a New Case is created and added to the duplicateFlags (case queue). The status of all involved voters in the voterDatabase is set to "Flagged."
- The Verifier UI reads from the duplicateFlags array to display pending cases.
- The Verifier submits a Resolution (the ID of the original voter).
- The Resolve Case function updates the voterDatabase, setting the original as "Verified" and all others as "Duplicate." It also adds an entry to the verificationHistory and removes the case from the duplicateFlags queue.

4.3 Use case Design

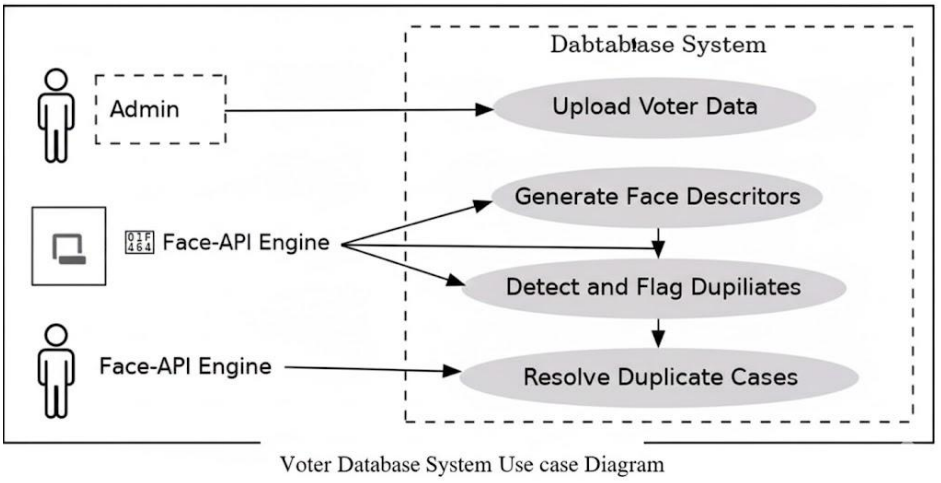


Fig 4.3: Use-case diagram of Sentinel

The Figure 4.3, The Voter Database System is designed to manage voter registration while automatically detecting potential duplicates using facial recognition. The process begins when the Admin uploads Voter Data (CSV and images), which the external Face-API Engine uses to Generate Face Descriptors. The system then automatically utilizes these descriptors to Detect and Flag Duplicates by comparing new data against existing records. If a match is found, the case is queued for human review. Finally, the Verifier reviews the potential duplicates and Resolves Duplicate Cases, ensuring the integrity of the voter database.

Chapter 5

IMPLEMENTATION

This chapter describes the project's development process and offers a thorough examination of how the key elements such as the logic and algorithms that drive the Sentinel system—are implemented.

5.1 Development Methodology

This project was built using an Iterative Methodology. This was essential because the core problem (facial matching) was experimental. Our initial assumptions were proven wrong, requiring us to refactor and improve the design.

- Iteration 1: Basic UI & Hashing: Created the HTML structure, CSS, and basic JavaScript logic. Implemented a simple cryptographic hash for images. Result: Failed. It could not match two different image files, even of the same person.
- Iteration 2: Perceptual Hashing: Replaced the crypto hash with a "Difference Hash" (dHash). This was better and could match resized/re-compressed images. Result: Failed. It could not match the same person from different angles.
- Iteration 3: AI Model Integration: The major breakthrough. We integrated face-api.js. This successfully matched people from different angles. However, the logic was a simple "auto-reject."
- Iteration 4: Case-Based Workflow (The "Pivot"): We realized auto-rejection was dangerous. We completely rewrote the logic to *flag* matches instead of rejecting them. This created the Admin/Verifier roles, the duplicateFlags array, and the "Case-Based" system, which is the core of the final project.
- Iteration 5: Performance & UI Polish: Optimized the "Process Batch" function to run in parallel (Promise.all), significantly speeding it up. Added the Admin history log and fixed UI bugs (like the object-fit: contain fix).

5.2 Implementation Details

The application is a single index.html file that links to style.css and script.js. All logic and data storage (in JavaScript arrays) runs in the client's browser. The face-api.js models are loaded

from a CDN on page load. The system is divided into two parts: the Admin's "processing pipeline" and the Verifier's "resolution workflow."

5.2.1 Technology Stack

- Front-End: HTML5, CSS3, "Vanilla" JavaScript (ES6+).
- Core Technology: face-api.js (a JavaScript library for face detection and recognition using browser-based TensorFlow.js).
- AI Models Used:
  - SsdMobilenetv1: A high-performance model for detecting the *location* of a face in an image.
  - faceLandmark68Net: A model to find the 68 specific points (eyes, nose, etc.) on the face.
  - faceRecognitionNet: A model that converts the 68 points into a 128-point "face descriptor" vector.
- Development Environment: VS Code with Live Server.

5.3 Core Logic and Algorithms (Pseudocode)

Algorithm 5.3.1: Image Processing (Parallel)

```
FUNCTION processBulkBtn_onClick:
  SET loadingSpinner to visible

  // Step 1: Create a promise for each image to be processed
  PROMISE_ARRAY = []
  FOR voter IN uploadedCSVData:
    FIND imageFile for voter
    IF imageFile is found:
      // Create a promise that generates the descriptor
      processingPromise = generateImageHash(imageFile)
      // Add promise and voter data to array
      PROMISE_ARRAY.push({ voter, processingPromise })
    END_IF
  END_FOR

  // Step 2: Run all promises in parallel


  processedVoters = WAIT FOR Promise.all(PROMISE_ARRAY.processingPromise)
  // Step 3: Now check them sequentially
  FOR i FROM 0 to processedVoters.length:
    newDescriptor = processedVoters[i].descriptor
```

```
        voterData = PROMISE_ARRAY[i].voter
        IF newDescriptor is valid:
            // Check against the *current* state of the database
            matchedVoters = checkForDuplicates(newDescriptor)

            IF matchedVoters.length == 0:
                // Unique: Add directly to DB
                ADD voterData to voterDatabase with status "verified"
            ELSE:
                // Duplicate: Create a case
                CREATE_CASE(newVoterData, matchedVoters)
            END_IF
        END_IF
    END_FOR
    SET loadingSpinner to hidden
    DISPLAY results
```

Algorithm 5.3.2: Generate Face Descriptor (The AI)

```
FUNCTION generateImageHash(imageFile):
    // This calls the face-api.js library
    img = CONVERT fileToImage(imageFile)

    detection = WAIT FOR faceapi.detectSingleFace(img, SsdMobilenetv1)
                            .withFaceLandmarks()
                            .withFaceDescriptor()

    IF detection is valid:
        RETURN detection.descriptor  // This is the 128-point vector
    ELSE:
        RETURN null
    END_IF
```

Algorithm 5.3.3: Check For Duplicates (The Logic)

```
FUNCTION checkForDuplicates(newDescriptor):
    matches = []
    DISTANCE_THRESHOLD = 0.4 // This is the similarity tolerance

    FOR voter IN voterDatabase:
        IF voter.faceDescriptor is valid:
            // Calculate distance between the two 128-point vectors
                    distance = faceapi.euclideanDistance(newDescriptor,
                                voter.faceDescriptor)

            IF distance < DISTANCE_THRESHOLD:
                ADD voter to matches
            END_IF
```

```
    END_IF
  END_FOR

  RETURN matches
```

Algorithm 5.3.4: Resolve Case (The Verifier's Action)

```
FUNCTION resolveCase(caseId, originalVoterId):
  // 1. Find the case in the Verifier's queue
  case = FIND case in duplicateFlags WHERE case.caseId == caseId

  // 2. Update all voters in the database
  FOR voter IN case.voters:
    dbEntry = FIND voter in voterDatabase
    IF voter.voterId == originalVoterId:
      dbEntry.status = "verified"
      dbEntry.caseId = null
    ELSE:
      dbEntry.status = "duplicate"
      dbEntry.caseId = null
    END_IF
  END_FOR

  // 3. Create history log
  historyEntry = {
    caseId: caseId,
    resolvedBy: currentUser.username,
    timestamp: new Date()
  }
  ADD historyEntry to verificationHistory

  // 4. Remove case from queue
  REMOVE case from duplicateFlags

  REFRESH_VERIFIER_UI()
  ALERT "Case Resolved"
```

Chapter 6

TESTING

6.1 Testing Overview

The testing phase was crucial for validating the end-to-end workflow, especially the accuracy of the AI model and the reliability of the case-based logic. We focused on ensuring that *no* data was lost and that the Verifier was *always* presented with the correct information.

6.2 Types of Testing

To ensure comprehensive coverage, several testing types were employed, each targeting a different aspect of the Sentinel system.

6.2.1 Unit Testing

Individual functions were tested in isolation. For example, generateImageHash was tested with a single image to ensure it returned a 128-float array. The resolveCase function was tested by manually creating a fake case and verifying that the voterDatabase was updated correctly.

6.2.2 Integration Testing

We tested the flow between components. A key test was uploading a new voter that matched an *already flagged* voter to ensure the new voter was correctly added to the existing case instead of creating a new one.

6.2.3 System Testing (End-to-End)

This was our primary test, simulating the exact user scenario:

1. Logged in as Admin.
2. Uploaded 6 voter entries in the CSV.
3. Uploaded 6 images (2 pics for Person A, 2 pics for Person B, 1 for Person C, 1 for Person D).
4. Clicked "Process."
5. Verified the Admin results showed "2 new unique entry" (Person C and D) and "2 new cases created."
6. Logged out and logged in as Verifier.
7. Verified the Verifier dashboard showed 2 cases: Case 1 (A), Case 2 (B).
8. Resolved all 2 cases.

9. Logged out and logged in as Admin.
10. Verified the Admin history log showed 2 new entries.

6.2.4 Usability Testing

This was performed on the UI. The most significant finding was that images were being cropped (object-fit: cover). This was fixed by changing the CSS to object-fit: contain to ensure the Verifier could see the entire face.

6.3 Test Cases

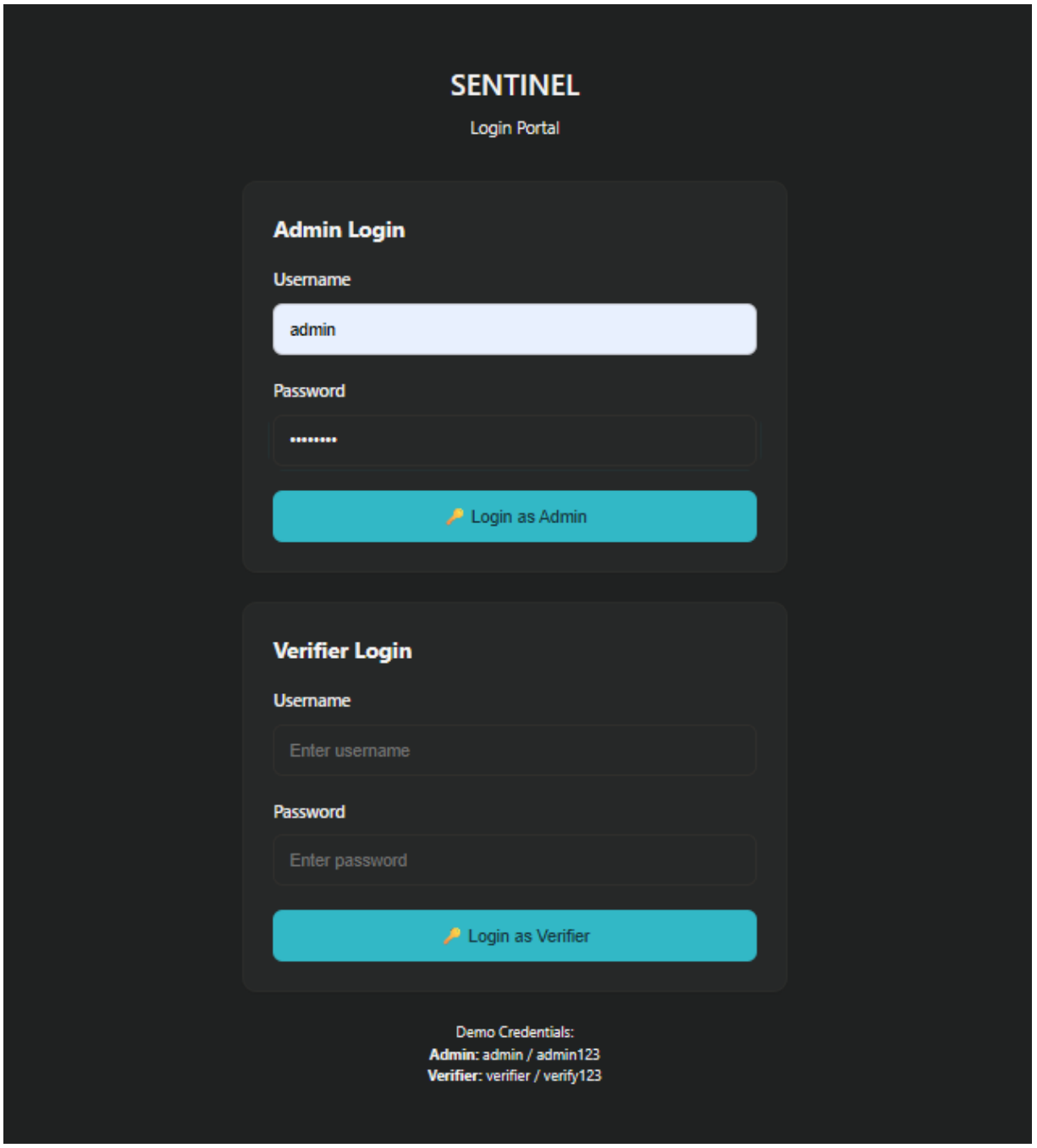
Test Case ID	Requirement / Use Case	Test Description	Expected Result	Actual Result
TC-001	FR-1	Try to log in with "admin" / "admin123".	Admin dashboard page is displayed.	**Pass**
TC-002	FR-1	Try to log in with "verifier" / "verify123".	Verifier dashboard page is displayed.	**Pass**
TC-003	FR-2, FR-3	Upload a CSV file and corresponding images.	"Process & Check Duplicates" button becomes enabled.	**Pass**
TC-004	FR-4, FR-5	Upload 6 images of 4 people (2, 3, 2, 1).	System processes and creates 2 new cases. 2 voters are auto-verified.	**Fail**
TC-005	FR-6	Log in as Verifier after TC-004.	Verifier dashboard shows 2 pending cases.	**Pass**
TC-006	FR-7, FR-8	Resolve a case with 2 voters. Mark 1 as original.	The case is removed from the queue. The voterDatabase is updated (1 "verified", 1 "duplicate").	**Pass**
TC-007	FR-9	Log in as Admin after TC-006.	The "Verifier Activity History" shows 1 new entry for the resolved case.	**Pass**
TC-008	NFR-3	Process a batch of 10 images.	The loading spinner is visible, and the browser does not freeze. (Parallel processing).	**Pass**
TC-009	NFR-4	View a case with a tall, narrow image.	The entire image is visible without cropping (object-fit: contain).	**Pass**

Table 6.3.1: Test cases

Table 6.3.1 summarizes the 9 key tests performed on the Voter ID Detector, confirming that all critical functions—like Admin/Verifier login, duplicate case generation, and Verifier resolution—passed successfully.

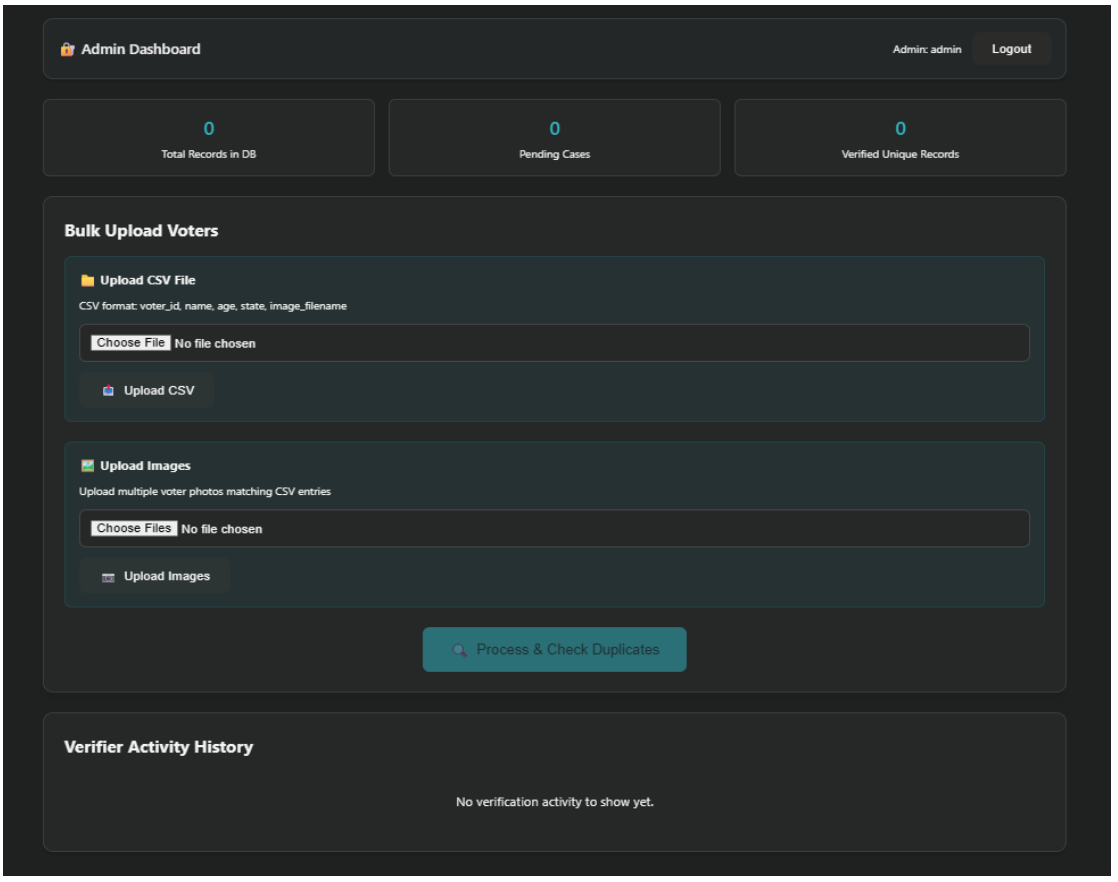
Chapter 7

SCREENSHOTS



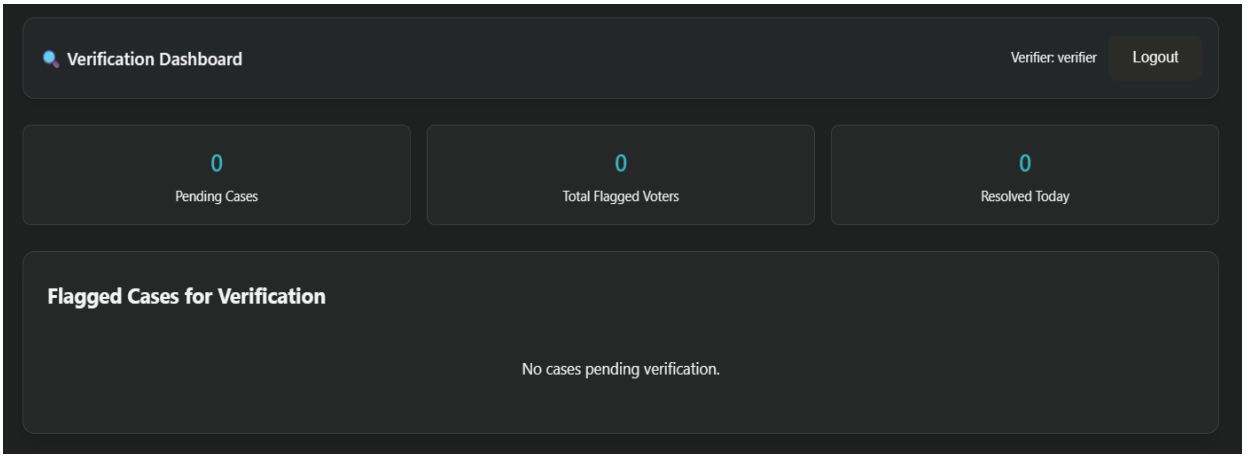
Screenshot 7.1 – Login Page: Sentinel

Screenshot 7.1: Login Page: A screenshot of the application's main entry point, showing the two distinct login forms for the "Admin" role and the "Verifier" role.



Screenshot 7.2 – Admin Dashboard and Upload Interface

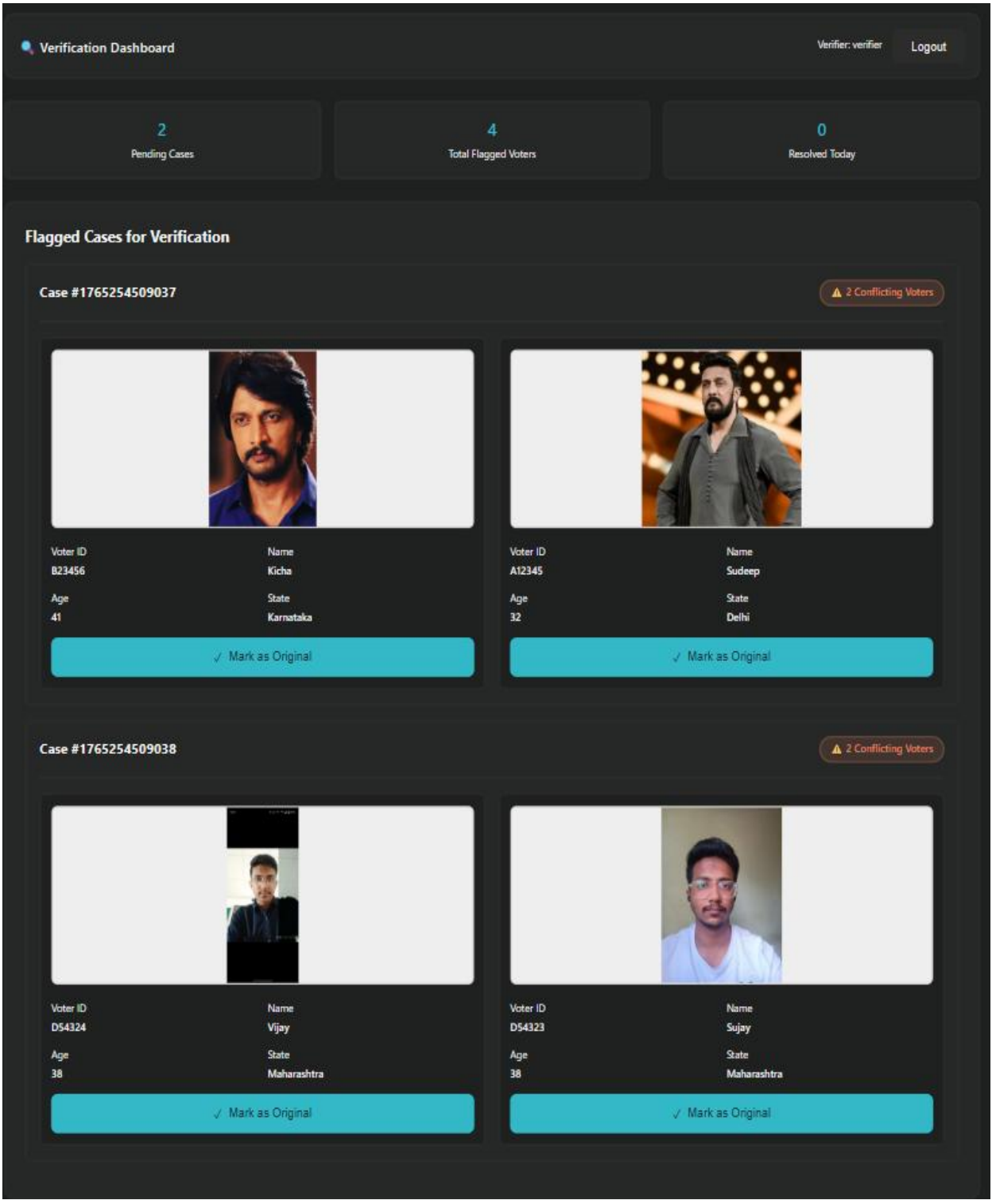
Screenshot 7.2: Admin Dashboard & Upload Interface A screenshot of the Admin's main dashboard after login. It displays the key statistics (Total Records, Pending Cases, Verified Records) and the interface for uploading the "Voter CSV File" and "Voter Images."



Screenshot 7.3 – Verification Dashboard with Pending Cases

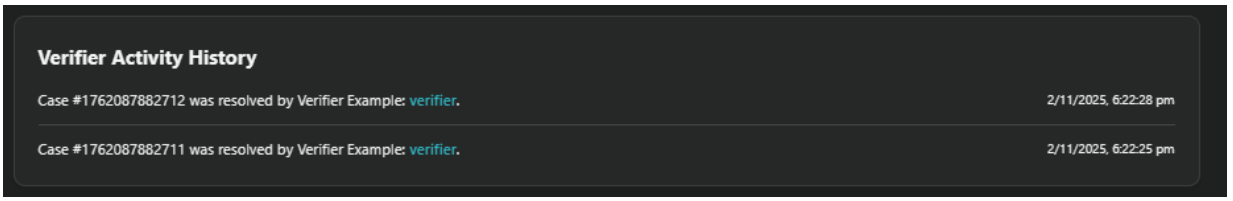
Screenshot 7.3: Verifier Dashboard with Pending Cases A screenshot of the Verifier's main dashboard, showing a queue of "Flagged Cases for Verification." Each "Case Card" represents

a cluster of potential duplicates awaiting review.



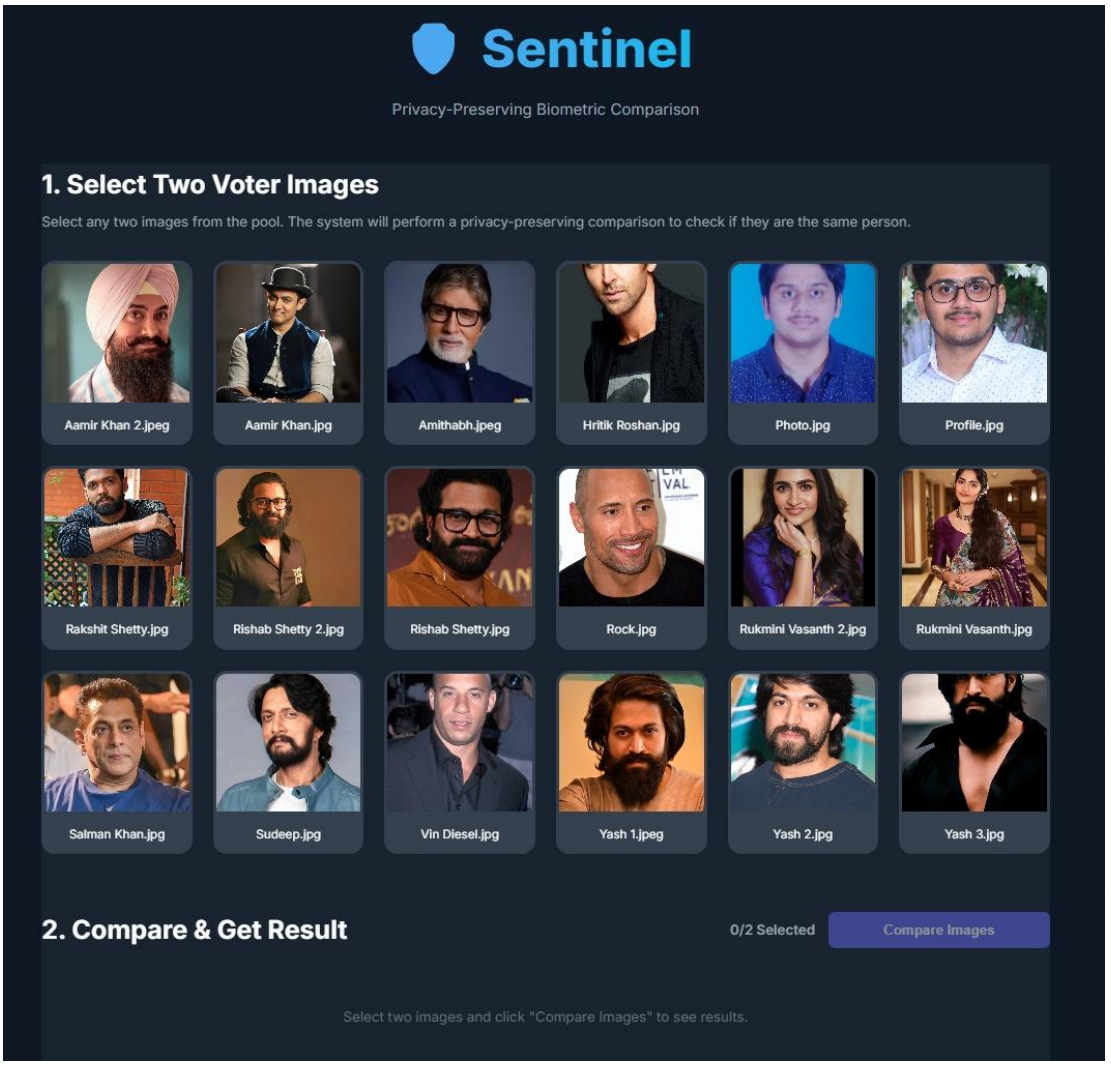
Screenshot 7.4 – Verifier Dashboard with Pending Cases

Screenshot 7.4: Verifier Case Review Interface A close-up screenshot of a single "Case Card" on the Verifier's dashboard. It clearly displays two or more conflicting voter entries side-by-side, showing their photos and data, with a "Mark as Original" button for each.



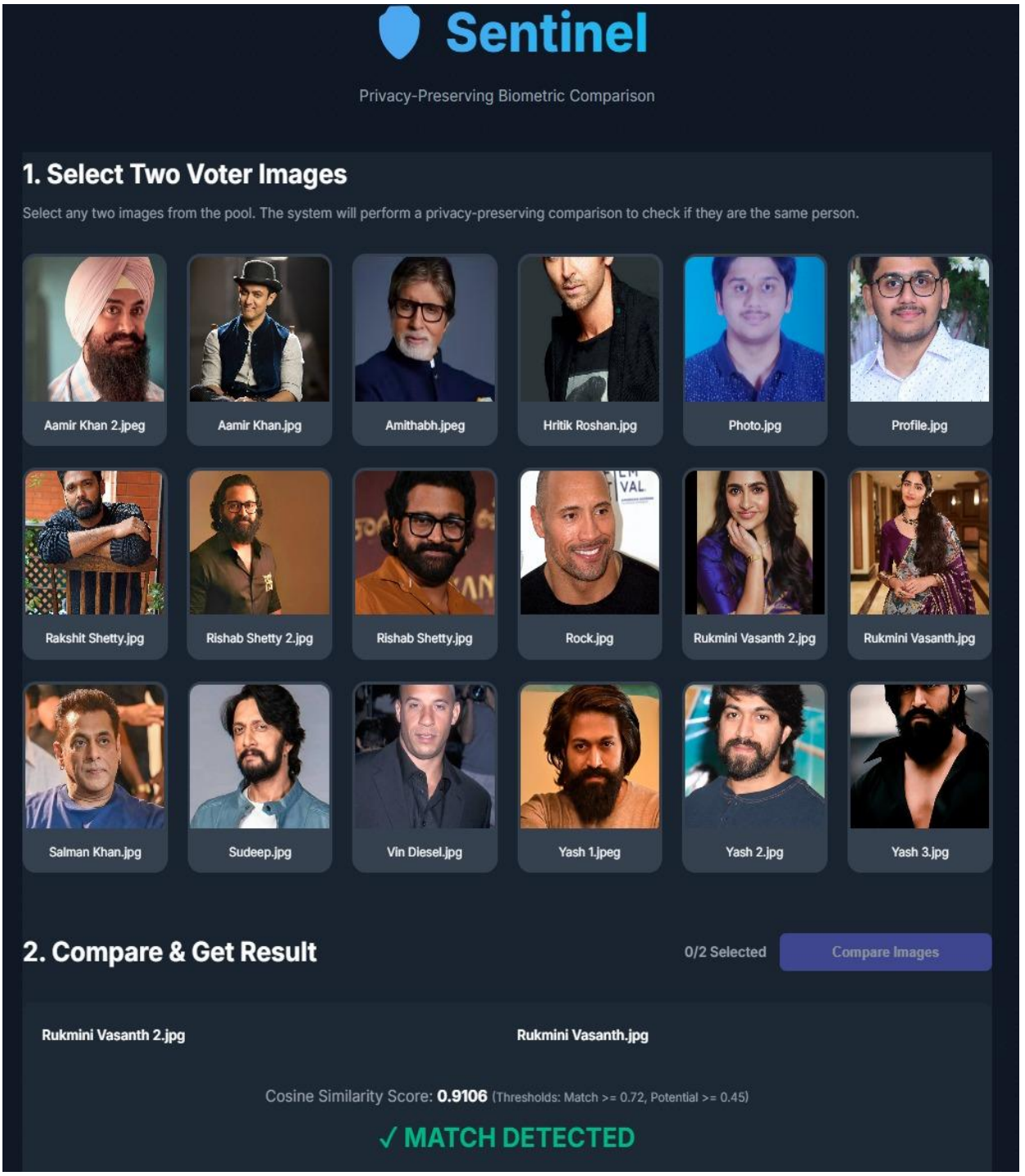
Screenshot 7.5 – Admin Activity History Log

Screenshot 7.5: Admin Activity History Log A screenshot of the Admin's dashboard, scrolled down to the "Verifier Activity History" section. It shows a list of recently resolved cases, including the Verifier's username, the Case ID, and the timestamp of the resolution.



Screenshot 7.6 – Biometric Test Harness

Screenshot 7.6: This screenshot shows the UI of the "Sentinel" prototype, a tool built to test the core biometric comparison logic. It displays the pool of test images and the "Compare & Get Result" section in its default state, before a comparison is run.



Screenshot 7.7 – Test Harness (Successful Match)

Screenshot 7.7: This screenshot shows the result of a successful 1-to-1 comparison. Two different images of the same person ("Rukmini Vasanth") were selected and compared. The system returned a "MATCH DETECTED" with a Cosine Similarity Score of 0.9106, (well above the 0.72 threshold), validating the algorithm's accuracy.

CONCLUSION

This is project successfully demonstrates a robust and reliable system for identifying duplicate voter registrations using cutting-edge biometric facial recognition technology. The Sentinel: True Identity Engine establishes a new standard for maintaining electoral roll accuracy.

By combining a powerful CNN-based model (implemented efficiently via face-api.js) with a transparent, case-based, human-in-the-loop workflow, the system directly addresses and solves the critical flaws inherent in legacy methods. Specifically, it overcomes the inaccuracy of simple cryptographic hashing (which fails on image variance) and mitigates the severe risk of wrongful disenfranchisement posed by fully-automated AI systems that lack human governance.

The architectural decision to enforce a strict separation of roles between the Admin (data management) and the Verifier (adjudication) creates an inherently secure and accountable process. This governance model is complemented by the system's core technical strengths: its ability to reliably match the same person from different angles and conditions (high accuracy) combined with its foundational policy of never auto-rejecting a match (guaranteed fairness).

The Voter ID Duplicate Detector serves as a strong proof-of-concept for how modern Artificial Intelligence and thoughtful User Experience (UX) design can be integrated to solve a critical, real-world problem in election integrity. Sentinel is not merely a technical solution; it is a governance tool that demonstrates that advanced biometric technology can be implemented in a manner that is both highly accurate and ethically compliant, offering a blueprint for scalable, trustworthy electoral roll management worldwide.

FUTURE ENHANCEMENTS

This project is a successful prototype, but for a real-world, large-scale deployment, the following enhancements would be necessary:

- Persistent Backend Database: The current "database" is a JavaScript array, which resets on page refresh. The immediate next step is to build a real backend (e.g., using Node.js and Express) and a persistent database (e.g., MongoDB or PostgreSQL) to store all voter data, face descriptors, and cases.
- Scalable, Server-Side AI Processing: The face-api.js library runs in the browser. This is slow and will crash if you try to process 10,000 voters. The AI processing should be moved to a Python backend server (using libraries like face_recognition or deepface) where it can be properly scaled.
- Secure Authentication: The current login is hardcoded. This must be replaced with a secure authentication system, such as JSON Web Tokens (JWT), to manage user sessions properly.
- Liveness Detection: A sophisticated user could trick the system by uploading a *photo of another person's photo*. A "Liveness Detection" model could be added, requiring the user to perform an action (like blink or turn their head) to prove they are a live person.
- Advanced Case Management: An enhanced Verifier dashboard could include tools for searching cases, filtering by date, and viewing the *similarity score* (the distance metric) for each match.

REFERENCES

1. (2024/2025). "Research on blockchain electronic voting system based on face recognition and deep fake face detection". *International Journal of Innovative Research in Scientific Scholarship (IJIRSS)*.
2. (2024). "Voter Authentication Using Enhanced ResNet50 for Facial Recognition". *MDPI*.
3. (2024). "E-Voting System Using Blockchain and Face Recognition". *International Research Journal of Engineering and Technology (IRJET)*.
4. (2025). "Application of human-in-the-loop hybrid augmented intelligence approach in security inspection system". *Frontiers in Artificial Intelligence*.
5. (2023). "Human-in-Loop: A Review of Smart Manufacturing Deployments". *MDPI*.
6. (2024). "The threat of artificial intelligence to elections worldwide: A review of the 2024 landscape". *World Journal of Advanced Engineering Technology and Sciences*.
7. Syafhendry et al. (2025). "Smart elections or rigged algorithms: the rise of artificial intelligence in electoral governance in Southeast Asia". *Frontiers in Political Science*.
8. (2025). "Research and Analysis of Facial Recognition Based on FaceNet, DeepFace, and OpenFace". *ITM Web of Conferences*.
9. Bönström, V. (2018). *face-api.js - JavaScript API for Face Recognition.* GitHub Repository.
10. Smirnov, D., et al. (2019). "TensorFlow.js: A JavaScript Library for ML in the Browser." *arXiv preprint arXiv:1901.05350*.
11. He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
12. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). "FaceNet: A Unified Embedding for Face Recognition and Clustering." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
13. Liu, W., et al. (2016). "SSD: Single Shot MultiBox Detector." *European Conference on Computer Vision (ECCV)*.
14. Howard, A. G., et al. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *arXiv preprint arXiv:1704.04861*.
15. MDN Web Docs. (2025). "HTML: HyperText Markup Language." *Mozilla Developer Network*. (https://developer.mozilla.org/en-US/docs/Web/HTML)
16. MDN Web Docs. (2025). "JavaScript." *Mozilla Developer Network*. (https://developer.mozilla.org/en-US/docs/Web/JavaScript)
17. W3C. (2020). "Cross-Origin Resource Sharing (CORS) Specification." *World Wide Web Consortium (W3C)*. (https://www.w3.org/TR/cors/)